

Practical: 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
29/07/2026	

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

Bubble Sort

Step-1: Start
Step-2: Read the number of elements n
Step-3: Compare adjacent elements
Step-4: If $arr[i] > arr[i+1]$, swap them
Step-5: Repeat for all elements in the array
Step-6: Repeat the process until the array is sorted
Step-7: Stop

Selection Sort

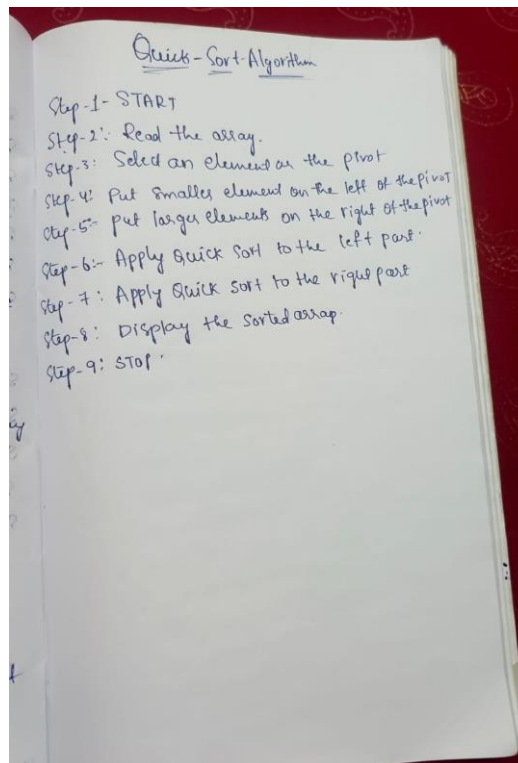
Step-1: START
Step-2: Read the number of elements n
Step-3: Read the array $arr[0]$ to $arr[n-1]$
Step-4: Set $min = 0$
Step-5: Repeat while $i < n-1$
 • Set min index to i
 • Repeat while $j < n-i$
 • If $arr[j] < arr[min]$, set $min = j$
 • Swap $arr[i]$ and $arr[min]$
 • Increment i
Step-6: Display the sorted array
Step-7: Stop

Insertion Sort Algorithm

1) Step-1: Start
Step-2: Read the number of elements n
Step-3: Read the array $arr[0]$ to $arr[n-1]$
Step-4: Set $i = 1$
Step-5: Repeat while $i < n$
 • Set $key = arr[i]$
 • Set key to $arr[i]$
 • Set $j = i-1$
 • Repeat while $j > 0$ and $arr[j] > key$
 • Set $arr[j+1] = arr[j]$
 • Set $arr[j] = key$
 • Increment i
Step-6: Increment, Display the sorted array
Step-7: Stop

Merge sort - Algorithm

Step-1: START
Step-2: Read the array
Step-3: Find the middle element
Step-4: Divide the array into two halves
Step-5: Apply Merge-Sort to the left half
Step-6: Apply Merge-Sort to the right half
Step-7: Merge the two sorted halves
Step-8: Display the sorted array



Program:

```
#include <iostream>
using namespace std;

// ----- BUBBLE SORT -----
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// ----- SELECTION SORT -----
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }
}

// ----- INSERTION SORT -----
void insertionSort(int arr[], int n)
{

```

```
for (int i = 1; i < n; i++)
{
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}

// ----- MERGE SORT -----
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];
    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];

    for (i = low; i <= high; i++)
        arr[i] = temp[i];
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

// ----- QUICK SORT -----
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
}
```

```
    }  
}  
  
swap(arr[i + 1], arr[high]);  
  
return i + 1;  
}  
  
void quickSort(int arr[], int low, int high)  
{  
    if (low < high)  
    {  
        int p = partition(arr, low, high);  
  
        quickSort(arr, low, p - 1);  
        quickSort(arr, p + 1, high);  
    }  
}  
  
// ----- DISPLAY -----  
void display(int arr[], int n)  
{  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
  
    cout << endl;  
}  
  
// ----- MAIN -----  
int main()  
{  
    int n;  
    int original[100];  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++)  
        cin >> original[i];  
  
    int bubble[100], selection[100], insertion[100];  
    int mergeArr[100], quick[100];  
  
    // Copy original array  
    for (int i = 0; i < n; i++)  
    {  
        bubble[i] = original[i];  
        selection[i] = original[i];  
        insertion[i] = original[i];  
        mergeArr[i] = original[i];  
        quick[i] = original[i];  
    }  
  
    // Sorting  
    bubbleSort(bubble, n);  
    selectionSort(selection, n);  
    insertionSort(insertion, n);  
    mergeSort(mergeArr, 0, n - 1);  
    quickSort(quick, 0, n - 1);  
  
    // Display results  
    cout << "\nBubble Sort:   ";  
    display(bubble, n);
```

```
cout << "Selection Sort: ";
display(selection, n);

cout << "Insertion Sort: ";
display(insertion, n);

cout << "Merge Sort: ";
display(mergeArr, n);

cout << "Quick Sort: ";
display(quick, n);

return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements: 64 25 12 22 11 90

Bubble Sort:    11 12 22 25 64 90
Selection Sort: 11 12 22 25 64 90
Insertion Sort: 11 12 22 25 64 90
Merge Sort:     11 12 22 25 64 90
Quick Sort:     11 12 22 25 64 90
```

Time Complexity:

Time Complexity:

1) Bubble Sort

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

$T(n) = n(n-1) + n(n-2) + \dots + n(1)$
 $T(n) = \sum_{i=1}^{n-1} n(n-i)$
 $T(n) = n \sum_{i=1}^{n-1} (n-i)$
 $T(n) = n \left[(n-1) + (n-2) + \dots + 1 \right]$
 $T(n) = n \left[\frac{(n-1)(n)}{2} \right]$
 $T(n) = \frac{n^2(n-1)}{2}$
 $T(n) = O(n^3)$

2) Selection Sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int minIndex = i;
        for (int j = i+1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
}
```

$T(n) = (n-1)(n-2) + (n-3) + \dots + 1$
 $T(n) = \sum_{i=1}^{n-1} (n-i)$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

3) Insertion Sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i-1;
        while (j > 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

$T(n) = n + (n-1) + (n-2) + \dots + 1$
 $T(n) = \sum_{i=1}^{n-1} (n-i)$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

4) Merge Sort

```
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid+1, high);
        merge(arr, low, mid, high);
    }
}
```

$T(n) = 2T(n/2) + n$
 $T(n) = 2[2T(n/4) + n/2] + n$
 $T(n) = 4T(n/4) + 2n$
 $T(n) = 8T(n/8) + 3n$
 $T(n) = 16T(n/16) + 4n$
 $T(n) = 2^k T(n/2^k) + kn$
 $n/2^k = 1$
 $2^k = n$
 $k = \log_2 n$
 $T(n) = n \log_2 n + n$
 $T(n) = O(n \log n)$

Quick-sort:

Quick-sort(arr[], int low, int high) {
 if (low < high) {
 int p = partition(a, low, high);
 quicksort(a, low, p-1);
 quicksort(a, p+1, high);
 }
}

$$T(n) = T(k) + T(n-k-1) + O(n)$$

$k \approx n/2$

$$T(n) = 2T(n/2) + n$$

$$T(n) = O(n \log n) \Rightarrow \text{Best case}$$

Worst case:

$$T(n) = T(n-1) + n$$

$$T(n) = 1 + 2 + 3 + \dots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection and Insertion Sort are easy to understand and implement, but they can take more time for large data. Merge Sort and Quick Sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the required output.